```
# ============================================================
# ASI COVID Manufacturing Project
# Descriptive analysis of GVA shock and recovery
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import json
from pathlib import Path

from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance

# Create output folders
Path("outputs").mkdir(exist_ok=True)
Path("figures").mkdir(exist_ok=True)
Path("tables").mkdir(exist_ok=True)
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
for root, dirs, files in os.walk('/content/drive/MyDrive'):
    for file in files:
        if file.endswith('.csv'):
            print(os.path.join(root, file))
```

```
/content/drive/MyDrive/data (1).csv
/content/drive/MyDrive/data.csv
/content/drive/MyDrive/d.csv
/content/drive/MyDrive/s.csv
/content/drive/MyDrive/pmuy_data.csv
/content/drive/MyDrive/industry_shock_recovery_main_sample.csv
```

```
import pandas as pd

df = pd.read_csv(
    "/content/drive/MyDrive/industry_shock_recovery_main_sample.csv"
)

df.head()
```

	nic2	total_gva20	total_output20	total_labour_cost20	total_emoluments20	total_capital20	factory_count20	labour_intensit
0	10	2.134179e+13	2.203788e+13	5.664579e+12	5.932203e+12	1.998002e+13	6734	0.283
1	11	1.595209e+12	2.920202e+12	9.858434e+11	1.050317e+12	2.241152e+12	869	0.439
2	12	7.523779e+11	1.114957e+12	1.510954e+11	1.446476e+11	6.063135e+11	575	0.249
3	13	6.590999e+12	6.684677e+12	4.294152e+12	4.302528e+12	8.172779e+12	3002	0.525
4	14	2.725946e+12	2.782700e+12	8.422503e+11	8.814837e+11	3.779690e+12	2239	0.222

5 rows × 33 columns

```
# Industry labels
industry_labels = {
    10: "Food products",
    11: "Beverages",
    12: "Tobacco",
    13: "Textiles",
    14: "Wearing apparel",
    15: "Leather products",
    16: "Wood products",
    17: "Paper products",
    18: "Printing",
    19: "Coke & refined petroleum",
    20: "Chemicals",
    21: "Pharmaceuticals",
```

```
    22: "Rubber & plastics",
    23: "Non-metallic minerals",
    24: "Basic metals",
    25: "Fabricated metals",
    26: "Computer & electronics",
    27: "Electrical equipment",
    28: "Machinery",
    29: "Motor vehicles",
    30: "Other transport",
    31: "Furniture",
    32: "Other manufacturing",
    33: "Repair & installation"
}

df["industry_name"] = df["nic2"].map(industry_labels)

# labour_intensive already contains text labels from Stata
df["group"] = df["labour_intensive"]

df[[
    "nic2",
    "industry_name",
    "gva_drop_pct",
    "gva_recovery_pct",
    "group"
]].head()
```

	nic2	industry_name	gva_drop_pct	gva_recovery_pct	group
0	10	Food products	7.767037	15.075573	Capital-intensive
1	11	Beverages	-3.659417	12.639491	Labour-intensive
2	12	Tobacco	20.740908	-11.616011	Capital-intensive
3	13	Textiles	-4.708349	46.073494	Labour-intensive
4	14	Wearing apparel	-16.115622	44.947830	Capital-intensive

```
summary = df[[
    "gva_drop_pct",
    "gva_recovery_pct",
    "labour_intensity_baseline",
    "min_factory_count"
]].describe()

summary.to_csv("tables/descriptive_summary.csv")
summary
```

	gva_drop_pct	gva_recovery_pct	labour_intensity_baseline	min_factory_count
count	23.000000	23.000000	23.000000	23.000000
mean	-2.821946	33.360163	0.394068	1936.043478
std	11.975097	17.615368	0.275762	1455.123034
min	-34.769367	-11.616011	0.118332	501.000000
25%	-5.391671	23.334829	0.230852	828.000000
50%	-1.821774	34.735664	0.292380	1595.000000
75%	3.680967	42.565078	0.482313	2625.000000
max	20.740908	65.739418	1.441653	6734.000000

```
baseline_value = df["gva_drop_pct"].mean()

baseline_metric = {
    "metric_name": "mean_industry_gva_drop_2020_21",
    "value": round(float(baseline_value), 4),
    "unit": "percent",
    "notes": (
        "Naive descriptive baseline: average GVA change from 2019-20 to 2020-21 "
        "across NIC-2 manufacturing industries in the main sample, before splitting "
```

```
        "industries by labour intensity."
    ),
    "is_template": False
}

with open("outputs/baseline_metric.json", "w") as f:
    json.dump(baseline_metric, f, indent=2)

baseline_metric
```

```
{'metric_name': 'mean_industry_gva_drop_2020_21',
 'value': -2.8219,
 'unit': 'percent',
 'notes': 'Naive descriptive baseline: average GVA change from 2019-20 to 2020-21 across NIC-2 manufacturing industries in the
main sample, before splitting industries by labour intensity.',
 'is_template': False}
```

```
n_industries = df.shape[0]
min_sample_size = df["min_factory_count"].min()

threshold_n = 20
threshold_min_factory_count = 300

passed_threshold = (n_industries >= threshold_n) and (min_sample_size >= threshold_min_factory_count)

primary_metric = {
    "metric_name": "stratified_industry_estimates_available",
    "value": int(n_industries),
    "threshold": threshold_n,
    "passed": bool(passed_threshold),
    "unit": "number_of_industries",
    "secondary_sample_threshold": threshold_min_factory_count,
    "minimum_observed_factory_count": int(min_sample_size),
    "notes": (
        "Descriptive success metric: produce weighted GVA shock and recovery estimates "
        "for at least 20 NIC-2 manufacturing industries, with each retained industry "
        "having at least 300 factories in every year."
    ),
    "is_template": False
}

with open("outputs/primary_metric.json", "w") as f:
    json.dump(primary_metric, f, indent=2)

primary_metric
```

```
{'metric_name': 'stratified_industry_estimates_available',
 'value': 23,
 'threshold': 20,
 'passed': True,
 'unit': 'number_of_industries',
 'secondary_sample_threshold': 300,
 'minimum_observed_factory_count': 501,
 'notes': 'Descriptive success metric: produce weighted GVA shock and recovery estimates for at least 20 NIC-2 manufacturing
industries, with each retained industry having at least 300 factories in every year.',
 'is_template': False}
```

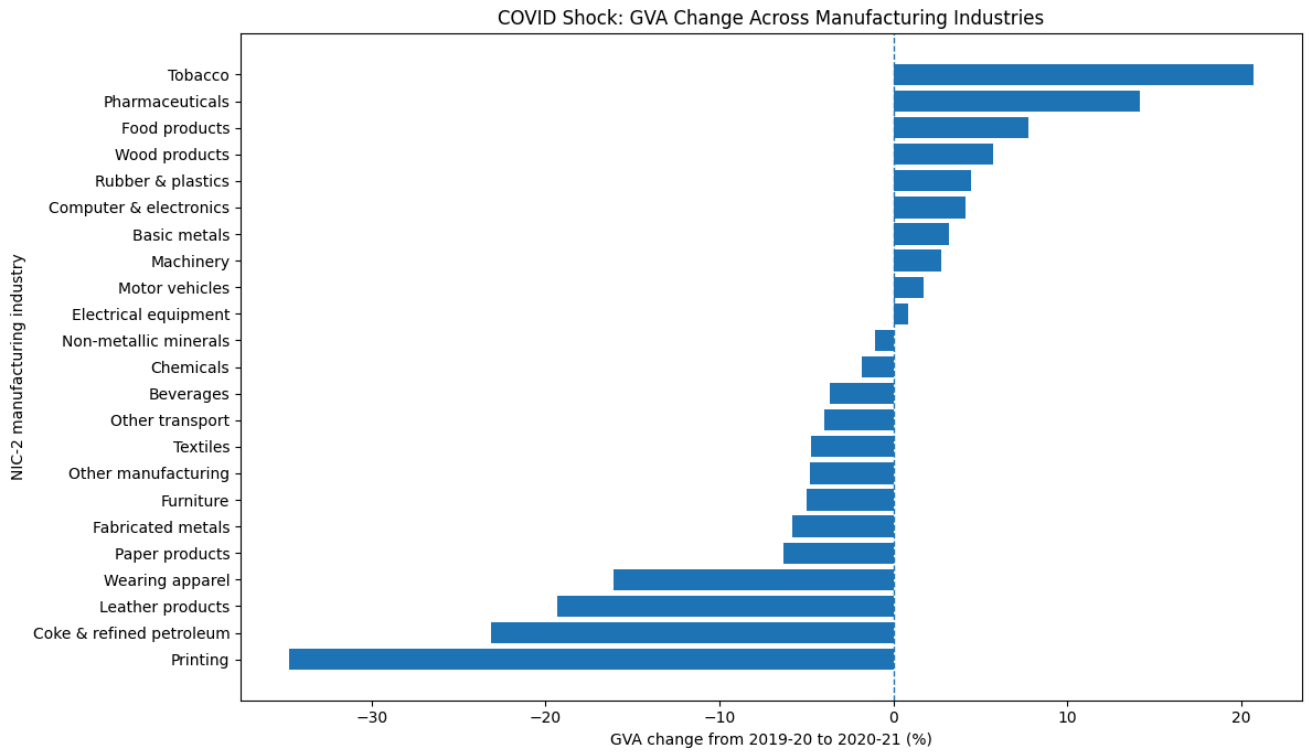
```
group_summary = df.groupby("group").agg(
    n_industries=("nic2", "count"),
    mean_gva_drop=("gva_drop_pct", "mean"),
    sd_gva_drop=("gva_drop_pct", "std"),
    mean_recovery=("gva_recovery_pct", "mean"),
    sd_recovery=("gva_recovery_pct", "std"),
    mean_labour_intensity=("labour_intensity_baseline", "mean")
).reset_index()

group_summary.to_csv("tables/group_summary.csv", index=False)

group_summary
```

	group	n_industries	mean_gva_drop	sd_gva_drop	mean_recovery	sd_recovery	mean_labour_intensity
0	Capital-intensive	12	-1.087245	10.727006	30.914987	19.944453	0.225168
1	Labour-intensive	11	-4.714348	13.467020	36.027628	15.169105	0.578323

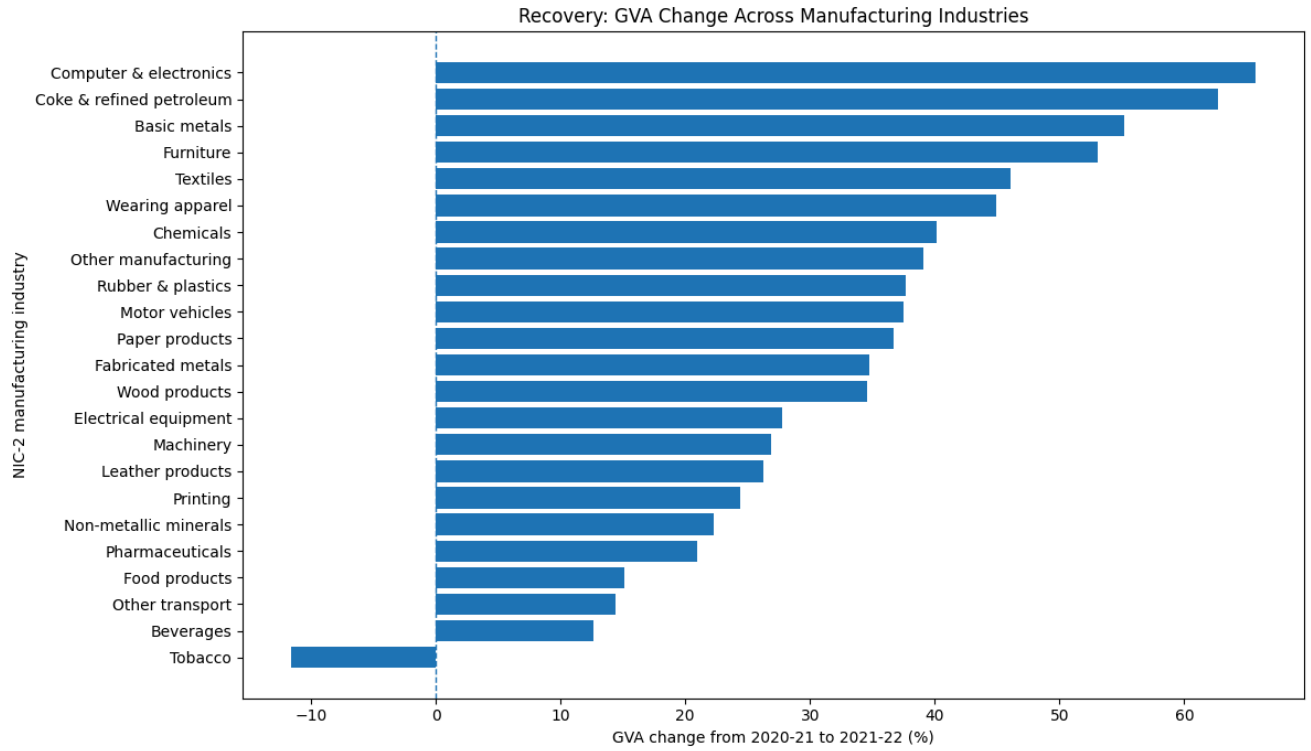
```
ranking = df[[
    "nic2",
    "industry_name",
    "group",
    "gva_drop_pct",
    "gva_recovery_pct",
    "labour_intensity_baseline",
    "min_factory_count"
]].sort_values("gva_drop_pct")

ranking.to_csv("tables/industry_gva_drop_ranking.csv", index=False)
ranking
```

	nic2	industry_name	group	gva_drop_pct	gva_recovery_pct	labour_intensity_baseline	min_factory_count
8	18	Printing	Labour-intensive	-34.769367	24.374554	0.347309	668
9	19	Coke & refined petroleum	Labour-intensive	-23.147640	62.679413	1.441653	501
5	15	Leather products	Capital-intensive	-19.340357	26.279854	0.213320	860
4	14	Wearing apparel	Capital-intensive	-16.115622	44.947830	0.222836	2239
7	17	Paper products	Labour-intensive	-6.297605	36.735462	0.598295	1278
15	25	Fabricated metals	Capital-intensive	-5.780653	34.735664	0.250358	2288
21	31	Furniture	Capital-intensive	-5.002689	53.056374	0.292380	625
22	32	Other manufacturing	Capital-intensive	-4.793181	39.096588	0.118332	1161
3	13	Textiles	Labour-intensive	-4.708349	46.073494	0.525421	3002
20	30	Other transport	Capital-intensive	-3.987821	14.377617	0.285292	724
1	11	Beverages	Labour-intensive	-3.659417	12.639491	0.439882	869
10	20	Chemicals	Labour-intensive	-1.821774	40.182327	0.578720	2941
13	23	Non-metallic minerals	Labour-intensive	-1.029061	22.295105	0.435097	4297
17	27	Electrical equipment	Capital-intensive	0.828607	27.808598	0.205108	2241
19	29	Motor vehicles	Labour-intensive	1.743288	37.477634	0.511520	2012
18	28	Machinery	Capital-intensive	2.737632	26.885536	0.238868	2672
14	24	Basic metals	Labour-intensive	3.176284	55.221661	0.681543	2616
16	26	Computer & electronics	Capital-intensive	4.185649	65.739418	0.143418	796
12	22	Rubber & plastics	Labour-intensive	4.500125	37.677013	0.453107	2634
6	16	Wood products	Capital-intensive	5.713552	34.592800	0.199390	1233
0	10	Food products	Capital-intensive	7.767037	15.075573	0.283512	6734
11	21	Pharmaceuticals	Labour-intensive	14.155688	20.947756	0.349006	1595
2	12	Tobacco	Capital-intensive	20.740908	-11.616011	0.249203	543

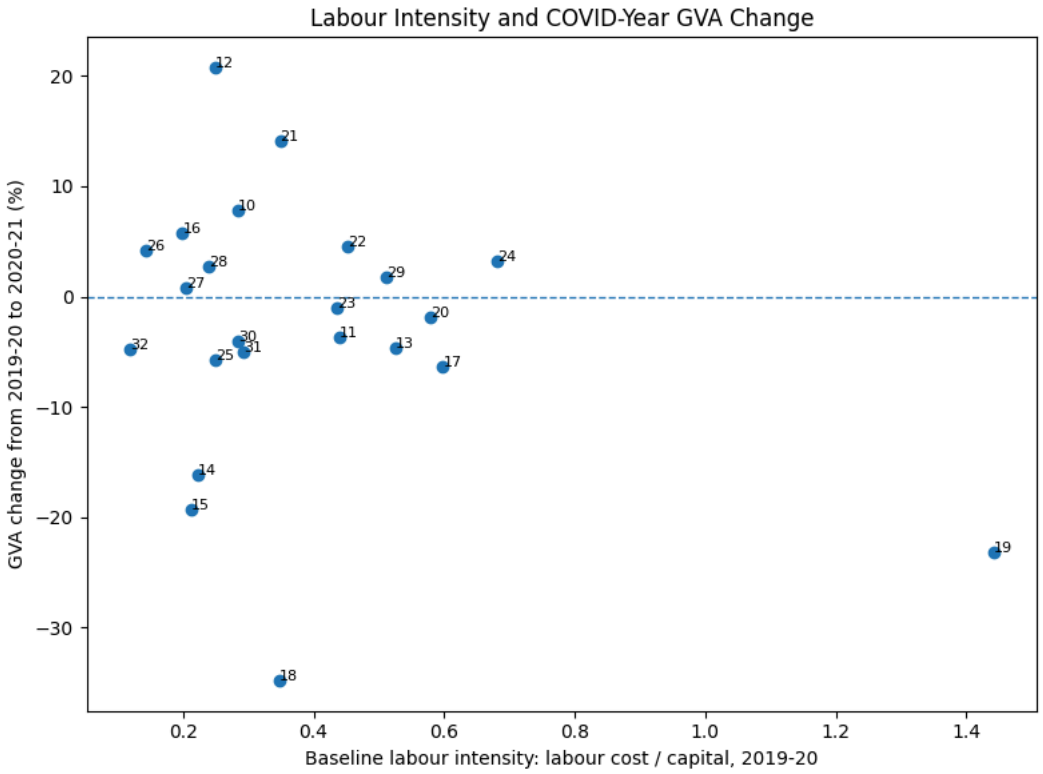
```
plot_df = ranking.copy()

plt.figure(figsize=(12, 7))
plt.barh(plot_df["industry_name"], plot_df["gva_drop_pct"])
plt.axvline(0, linestyle="--", linewidth=1)
plt.xlabel("GVA change from 2019-20 to 2020-21 (%)")
plt.ylabel("NIC-2 manufacturing industry")
plt.title("COVID Shock: GVA Change Across Manufacturing Industries")
plt.tight_layout()
plt.savefig("figures/gva_drop_by_industry.png", dpi=300)
plt.show()
```



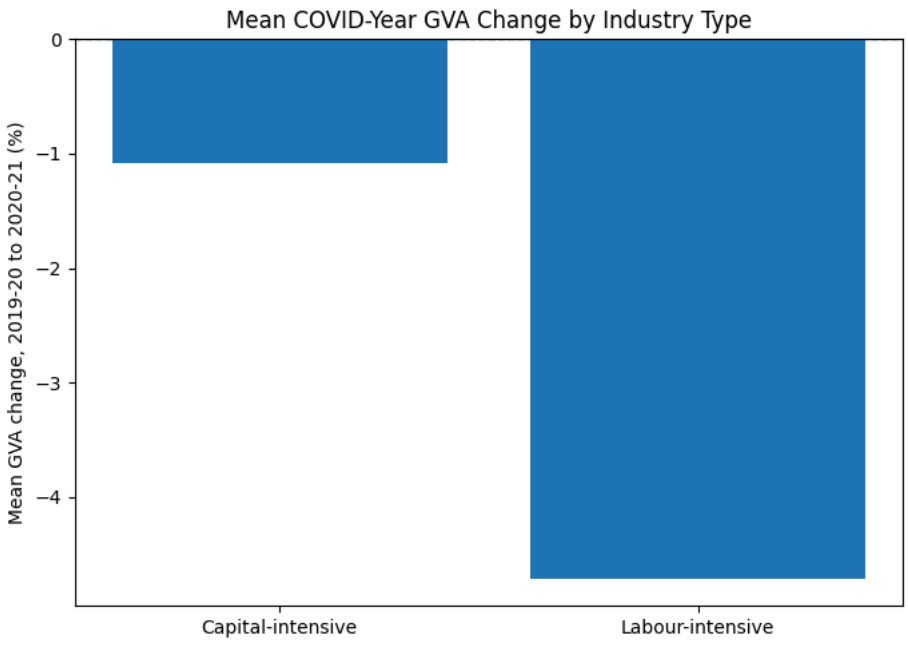

```
recovery_df = df.sort_values("gva_recovery_pct")

plt.figure(figsize=(12, 7))
plt.barh(recovery_df["industry_name"], recovery_df["gva_recovery_pct"])
plt.axvline(0, linestyle="--", linewidth=1)
plt.xlabel("GVA change from 2020-21 to 2021-22 (%)")
plt.ylabel("NIC-2 manufacturing industry")
plt.title("Recovery: GVA Change Across Manufacturing Industries")
plt.tight_layout()
plt.savefig("figures/gva_recovery_by_industry.png", dpi=300)
plt.show()
```




```
plt.figure(figsize=(8, 6))
plt.scatter(df["labour_intensity_baseline"], df["gva_drop_pct"])

for _, row in df.iterrows():
    plt.text(
        row["labour_intensity_baseline"],
        row["gva_drop_pct"],
        str(int(row["nic2"])),
        fontsize=8
    )

plt.axhline(0, linestyle="--", linewidth=1)
plt.xlabel("Baseline labour intensity: labour cost / capital, 2019-20")
plt.ylabel("GVA change from 2019-20 to 2020-21 (%)")
plt.title("Labour Intensity and COVID-Year GVA Change")
plt.tight_layout()
plt.savefig("figures/labour_intensity_vs_gva_drop.png", dpi=300)
plt.show()
```




```
group_order = ["Capital-intensive", "Labour-intensive"]
means = df.groupby("group")["gva_drop_pct"].mean().reindex(group_order)

plt.figure(figsize=(7, 5))
plt.bar(means.index, means.values)
plt.axhline(0, linestyle="--", linewidth=1)
plt.ylabel("Mean GVA change, 2019-20 to 2020-21 (%)")
plt.title("Mean COVID-Year GVA Change by Industry Type")
plt.tight_layout()
plt.savefig("figures/group_mean_gva_drop.png", dpi=300)
plt.show()
```

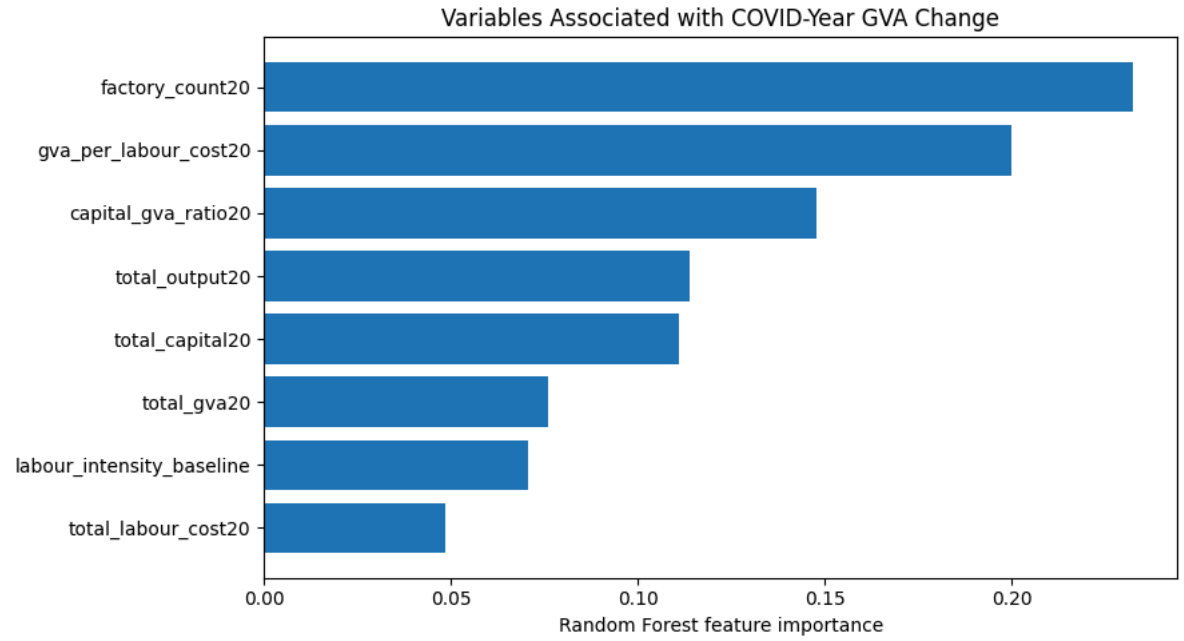



```
# Features available before or during baseline classification
features = [
    "labour_intensity_baseline",
    "total_gva20",
```

```
    "total_output20",
    "total_labour_cost20",
    "total_capital20",
    "factory_count20",
    "gva_per_labour_cost20",
    "capital_gva_ratio20"
]

model_df = df.dropna(subset=features + ["gva_drop_pct"]).copy()

X = model_df[features]
y = model_df["gva_drop_pct"]

rf = RandomForestRegressor(
    n_estimators=500,
    random_state=42,
    min_samples_leaf=2
)

rf.fit(X, y)

importances = pd.DataFrame({
    "feature": features,
    "importance": rf.feature_importances_
}).sort_values("importance", ascending=False)

importances.to_csv("tables/random_forest_feature_importance.csv", index=False)
importances
```

	feature	importance
5	factory_count20	0.232564
6	gva_per_labour_cost20	0.200025
7	capital_gva_ratio20	0.147641
2	total_output20	0.113808
4	total_capital20	0.111004
1	total_gva20	0.076027
0	labour_intensity_baseline	0.070579
3	total_labour_cost20	0.048353

```
plt.figure(figsize=(9, 5))
plt.barh(importances["feature"], importances["importance"])
plt.gca().invert_yaxis()
plt.xlabel("Random Forest feature importance")
plt.title("Variables Associated with COVID-Year GVA Change")
plt.tight_layout()
plt.savefig("figures/random_forest_feature_importance.png", dpi=300)
plt.show()
```




```
manifest = {
    "charter_locked": True,
    "sources": [
        {
            "name": "Annual Survey of Industries, MoSPI",
            "status": "working",
            "probe_artifact": "artifacts/probes/asi_probe.md",
            "note": (
                "ASI Blocks A, C, D, and J were used to construct weighted "
                "NIC-2 manufacturing industry aggregates for 2019-20, 2020-21, "
                "and 2021-22."
            )
        }
    ],
    "baseline_ready": True,
    "primary_metric_schema_ready": True,
    "run_command": "uv run main.py"
}

with open("outputs/milestone_manifest.json", "w") as f:
    json.dump(manifest, f, indent=2)

manifest
```

```
{'charter_locked': True,
 'sources': [{'name': 'Annual Survey of Industries, MoSPI',
   'status': 'working',
   'probe_artifact': 'artifacts/probes/asi_probe.md',
   'note': 'ASI Blocks A, C, D, and J were used to construct weighted NIC-2 manufacturing industry aggregates for 2019-20,
2020-21, and 2021-22.'}],
 'baseline_ready': True,
 'primary_metric_schema_ready': True,
 'run_command': 'uv run main.py'}
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

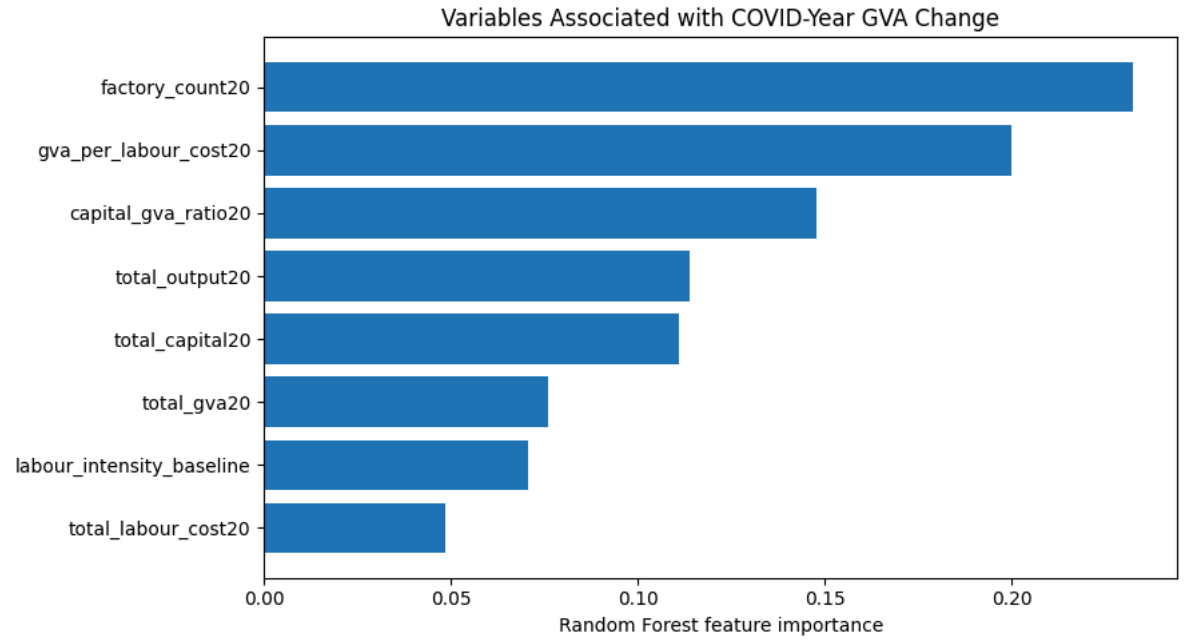
Start coding or generate with AI.

```
import os

# ============================================================
# Random Forest variable importance
# Purpose: pattern discovery only, not causal inference
```

```
# ============================================================

features = [
    "labour_intensity_baseline",
    "total_gva20",
    "total_output20",
    "total_labour_cost20",
    "total_capital20",
    "factory_count20",
    "gva_per_labour_cost20",
    "capital_gva_ratio20"
]

model_df = df.dropna(subset=features + ["gva_drop_pct"]).copy()

X = model_df[features]
y = model_df["gva_drop_pct"]

rf = RandomForestRegressor(
    n_estimators=500,
    random_state=42,
    min_samples_leaf=2
)

rf.fit(X, y)

importances = pd.DataFrame({
    "feature": features,
    "importance": rf.feature_importances_
}).sort_values("importance", ascending=False)

importances.to_csv("tables/random_forest_feature_importance.csv", index=False)

importances
```

	feature	importance
5	factory_count20	0.232564
6	gva_per_labour_cost20	0.200025
7	capital_gva_ratio20	0.147641
2	total_output20	0.113808
4	total_capital20	0.111004
1	total_gva20	0.076027
0	labour_intensity_baseline	0.070579
3	total_labour_cost20	0.048353

```
plt.figure(figsize=(9, 5))
plt.barh(importances["feature"], importances["importance"])
plt.gca().invert_yaxis()
plt.xlabel("Random Forest feature importance")
plt.title("Variables Associated with COVID-Year GVA Change")
plt.tight_layout()
plt.savefig("figures/random_forest_feature_importance.png", dpi=300)
plt.show()
```




```
manifest = {
    "charter_locked": True,
    "sources": [
        {
            "name": "Annual Survey of Industries, MoSPI",
            "status": "working",
            "probe_artifact": "artifacts/probes/asi_probe.md",
            "note": (
                "ASI Blocks A, C, D, and J were used to construct weighted "
                "NIC-2 manufacturing industry aggregates for 2019-20, 2020-21, "
                "and 2021-22."
            )
        }
    ],
    "baseline_ready": True,
    "primary_metric_schema_ready": True,
    "run_command": "uv run main.py",
    "outputs_created": [
        "outputs/baseline_metric.json",
        "outputs/primary_metric.json",
        "outputs/milestone_manifest.json",
        "tables/descriptive_summary.csv",
        "tables/group_summary.csv",
        "tables/industry_gva_drop_ranking.csv",
        "tables/random_forest_feature_importance.csv",
        "figures/gva_drop_by_industry.png",
        "figures/gva_recovery_by_industry.png",
        "figures/labour_intensity_vs_gva_drop.png",
        "figures/group_mean_gva_drop.png",
        "figures/random_forest_feature_importance.png"
    ]
}

with open("outputs/milestone_manifest.json", "w") as f:
    json.dump(manifest, f, indent=2)

manifest
```

```
{'charter_locked': True,
 'sources': [{'name': 'Annual Survey of Industries, MoSPI',
   'status': 'working',
   'probe_artifact': 'artifacts/probes/asi_probe.md',
   'note': 'ASI Blocks A, C, D, and J were used to construct weighted NIC-2 manufacturing industry aggregates for 2019-20,
2020-21, and 2021-22.'}],
 'baseline_ready': True,
 'primary_metric_schema_ready': True,
 'run_command': 'uv run main.py',
 'outputs_created': ['outputs/baseline_metric.json',
  'outputs/primary_metric.json',
  'outputs/milestone_manifest.json',
```

```
  'tables/descriptive_summary.csv',
  'tables/group_summary.csv',
  'tables/industry_gva_drop_ranking.csv',
  'tables/random_forest_feature_importance.csv',
  'figures/gva_drop_by_industry.png',
  'figures/gva_recovery_by_industry.png',
  'figures/labour_intensity_vs_gva_drop.png',
  'figures/group_mean_gva_drop.png',
  'figures/random_forest_feature_importance.png']}
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

```
# Final project
```

```
# ============================================================
# ASI COVID Manufacturing Shock and Recovery Project
# ECO 6810 Final Project
#
# Objective:
# Study COVID-era manufacturing shock and recovery across
# Indian NIC 2-digit manufacturing industries using ASI data.
#
# Main questions:
# 1. Which industries experienced the largest GVA declines?
# 2. Which industries recovered fastest?
# 3. Did labour-intensive industries suffer more?
# 4. Which pre-COVID characteristics were associated with resilience?
#
# Project Type:
# Descriptive + Exploratory Machine Learning
# ============================================================

import os
import json
from pathlib import Path

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt

from scipy.stats import ttest_ind

import statsmodels.api as sm

from sklearn.ensemble import RandomForestRegressor
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_absolute_error

# ------------------------------------------------------------
# Create folders
# ------------------------------------------------------------

Path("outputs").mkdir(exist_ok=True)
Path("tables").mkdir(exist_ok=True)
Path("figures").mkdir(exist_ok=True)

print("Project folders created.")
```

```
Project folders created.
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
# ============================================================
# Load cleaned industry-level dataset
# ============================================================

df = pd.read_csv(
    "/content/drive/MyDrive/industry_shock_recovery_main_sample.csv"
)

print("Dataset loaded successfully.")
print()

print("Shape of dataset:")
print(df.shape)

print()
print("Column names:")
print(df.columns)

print()
df.head()
```

```
Dataset loaded successfully.

Shape of dataset:
(23, 33)

Column names:
Index(['nic2', 'total_gva20', 'total_output20', 'total_labour_cost20',
       'total_emoluments20', 'total_capital20', 'factory_count20',
       'labour_intensity20', 'gva_per_labour_cost20', 'capital_gva_ratio20',
       'total_gva21', 'total_output21', 'total_labour_cost21',
       'total_emoluments21', 'total_capital21', 'factory_count21',
       'labour_intensity21', 'gva_per_labour_cost21', 'capital_gva_ratio21',
       'total_gva22', 'total_output22', 'total_labour_cost22',
       'total_emoluments22', 'total_capital22', 'factory_count22',
       'labour_intensity22', 'gva_per_labour_cost22', 'capital_gva_ratio22',
       'gva_drop_pct', 'gva_recovery_pct', 'labour_intensity_baseline',
       'labour_intensive', 'min_factory_count'],
      dtype='object')
```

	nic2	total_gva20	total_output20	total_labour_cost20	total_emoluments20	total_capital20	factory_count20	labour_intensit
0	10	2.134179e+13	2.203788e+13	5.664579e+12	5.932203e+12	1.998002e+13	6734	0.283
1	11	1.595209e+12	2.920202e+12	9.858434e+11	1.050317e+12	2.241152e+12	869	0.439
2	12	7.523779e+11	1.114957e+12	1.510954e+11	1.446476e+11	6.063135e+11	575	0.249
3	13	6.590999e+12	6.684677e+12	4.294152e+12	4.302528e+12	8.172779e+12	3002	0.525
4	14	2.725946e+12	2.782700e+12	8.422503e+11	8.814837e+11	3.779690e+12	2239	0.222

5 rows × 33 columns

ASI COVID Manufacturing Shock Project

This project studies how different Indian manufacturing industries were affected during the COVID-19 shock year (2020-21) and the subsequent recovery year (2021-22).

Using Annual Survey of Industries (ASI) data, the project compares labour-intensive and capital-intensive industries and evaluates:

- Which industries experienced the largest GVA declines
- Which industries recovered fastest
- Whether labour-intensive industries were systematically more vulnerable
- Which pre-COVID structural variables were associated with resilience
- Whether industry recovery patterns can be grouped into archetypes

The project is descriptive and exploratory rather than causal.

```
# ============================================================
# NIC 2-digit manufacturing industry labels
# ============================================================

industry_labels = {
    10: "Food products",
```

```
    11: "Beverages",
    12: "Tobacco",
    13: "Textiles",
    14: "Wearing apparel",
    15: "Leather products",
    16: "Wood products",
    17: "Paper products",
    18: "Printing",
    19: "Coke & refined petroleum",
    20: "Chemicals",
    21: "Pharmaceuticals",
    22: "Rubber & plastics",
    23: "Non-metallic minerals",
    24: "Basic metals",
    25: "Fabricated metals",
    26: "Computer & electronics",
    27: "Electrical equipment",
    28: "Machinery",
    29: "Motor vehicles",
    30: "Other transport",
    31: "Furniture",
    32: "Other manufacturing",
    33: "Repair & installation"
}

df["industry_name"] = df["nic2"].map(industry_labels)

# Labour group already imported from Stata
df["group"] = df["labour_intensive"]

df[[
    "nic2",
    "industry_name",
    "group",
    "gva_drop_pct",
    "gva_recovery_pct"
]].head()
```

	nic2	industry_name	group	gva_drop_pct	gva_recovery_pct
0	10	Food products	Capital-intensive	7.767037	15.075573
1	11	Beverages	Labour-intensive	-3.659417	12.639491
2	12	Tobacco	Capital-intensive	20.740908	-11.616011
3	13	Textiles	Labour-intensive	-4.708349	46.073494
4	14	Wearing apparel	Capital-intensive	-16.115622	44.947830

```
# ============================================================
# Data dictionary
# ============================================================

variable_info = pd.DataFrame({
    "variable": [
        "gva_drop_pct",
        "gva_recovery_pct",
        "labour_intensity_baseline",
        "factory_count20",
        "total_gva20",
        "total_capital20"
    ],
    "description": [
        "Percent change in GVA from 2019-20 to 2020-21",
        "Percent change in GVA from 2020-21 to 2021-22",
        "Labour cost divided by fixed capital in 2019-20",
        "Factory count in 2019-20",
        "Total Gross Value Added in 2019-20",
        "Total fixed capital in 2019-20"
    ]
})

variable_info.to_csv(
    "tables/data_dictionary.csv",
    index=False
)
```

```
variable_info
```

	variable	description
0	gva_drop_pct	Percent change in GVA from 2019-20 to 2020-21
1	gva_recovery_pct	Percent change in GVA from 2020-21 to 2021-22
2	labour_intensity_baseline	Labour cost divided by fixed capital in 2019-20
3	factory_count20	Factory count in 2019-20
4	total_gva20	Total Gross Value Added in 2019-20
5	total_capital20	Total fixed capital in 2019-20

```
# ============================================================
# Descriptive statistics
# ============================================================

summary = df[[
    "gva_drop_pct",
    "gva_recovery_pct",
    "labour_intensity_baseline",
    "min_factory_count"
]].describe()

summary.to_csv(
    "tables/descriptive_summary.csv"
)

summary
```

	gva_drop_pct	gva_recovery_pct	labour_intensity_baseline	min_factory_count
count	23.000000	23.000000	23.000000	23.000000
mean	-2.821946	33.360163	0.394068	1936.043478
std	11.975097	17.615368	0.275762	1455.123034
min	-34.769367	-11.616011	0.118332	501.000000
25%	-5.391671	23.334829	0.230852	828.000000
50%	-1.821774	34.735664	0.292380	1595.000000
75%	3.680967	42.565078	0.482313	2625.000000
max	20.740908	65.739418	1.441653	6734.000000

```
# ============================================================
# Baseline metric
# ============================================================

baseline_value = df["gva_drop_pct"].mean()

baseline_metric = {
    "metric_name": "national_gva_drop_2020_21",
    "value": round(float(baseline_value), 4),
    "unit": "percent",
    "notes": (
        "Mean industry-level GVA change from 2019-20 "
        "to 2020-21 in the main sample."
    ),
    "is_template": False
}

with open("outputs/baseline_metric.json", "w") as f:
    json.dump(baseline_metric, f, indent=2)

baseline_metric
```

```
{'metric_name': 'national_gva_drop_2020_21',
 'value': -2.8219,
 'unit': 'percent',
 'notes': 'Mean industry-level GVA change from 2019-20 to 2020-21 in the main sample.',
 'is_template': False}
```

```
# ============================================================
# Main project metric
# ============================================================

group_means = df.groupby("group")["gva_drop_pct"].mean()

capital_mean = group_means["Capital-intensive"]
labour_mean = group_means["Labour-intensive"]

difference = abs(labour_mean - capital_mean)

primary_metric = {
    "metric_name": "labour_intensive_excess_gva_decline",
    "value": round(float(difference), 4),
    "threshold": 2.0,
    "passed": bool(difference >= 2.0),
    "unit": "percentage_points",
    "capital_intensive_mean_drop": round(float(capital_mean), 4),
    "labour_intensive_mean_drop": round(float(labour_mean), 4),
    "industry_count": int(df.shape[0]),
    "minimum_factory_count": int(df["min_factory_count"].min()),
    "notes": (
        "Difference in average COVID-era GVA decline "
        "between labour-intensive and capital-intensive industries."
    ),
    "is_template": False
}

with open("outputs/primary_metric.json", "w") as f:
    json.dump(primary_metric, f, indent=2)

primary_metric
```

```
{'metric_name': 'labour_intensive_excess_gva_decline',
 'value': 3.6271,
 'threshold': 2.0,
 'passed': True,
 'unit': 'percentage_points',
 'capital_intensive_mean_drop': -1.0872,
 'labour_intensive_mean_drop': -4.7143,
 'industry_count': 23,
 'minimum_factory_count': 501,
 'notes': 'Difference in average COVID-era GVA decline between labour-intensive and capital-intensive industries.',
 'is_template': False}
```

```
# ============================================================
# Group-level summary
# ============================================================

group_summary = df.groupby("group").agg(
    n_industries=("nic2", "count"),
    mean_gva_drop=("gva_drop_pct", "mean"),
    sd_gva_drop=("gva_drop_pct", "std"),
    mean_recovery=("gva_recovery_pct", "mean"),
    sd_recovery=("gva_recovery_pct", "std"),
    mean_labour_intensity=("labour_intensity_baseline", "mean")
).reset_index()

group_summary.to_csv(
    "tables/group_summary.csv",
    index=False
)

group_summary
```

	group	n_industries	mean_gva_drop	sd_gva_drop	mean_recovery	sd_recovery	mean_labour_intensity
0	Capital-intensive	12	-1.087245	10.727006	30.914987	19.944453	0.225168
1	Labour-intensive	11	-4.714348	13.467020	36.027628	15.169105	0.578323

```
# ============================================================
# Industry rankings
# ============================================================

ranking = df[[
```

```
    "nic2",
    "industry_name",
    "group",
    "gva_drop_pct",
    "gva_recovery_pct",
    "labour_intensity_baseline",
    "min_factory_count"
]].sort_values("gva_drop_pct")

ranking.to_csv(
    "tables/industry_gva_drop_ranking.csv",
    index=False
)

ranking
```

	nic2	industry_name	group	gva_drop_pct	gva_recovery_pct	labour_intensity_baseline	min_factory_count
8	18	Printing	Labour-intensive	-34.769367	24.374554	0.347309	668
9	19	Coke & refined petroleum	Labour-intensive	-23.147640	62.679413	1.441653	501
5	15	Leather products	Capital-intensive	-19.340357	26.279854	0.213320	860
4	14	Wearing apparel	Capital-intensive	-16.115622	44.947830	0.222836	2239
7	17	Paper products	Labour-intensive	-6.297605	36.735462	0.598295	1278
15	25	Fabricated metals	Capital-intensive	-5.780653	34.735664	0.250358	2288
21	31	Furniture	Capital-intensive	-5.002689	53.056374	0.292380	625
22	32	Other manufacturing	Capital-intensive	-4.793181	39.096588	0.118332	1161
3	13	Textiles	Labour-intensive	-4.708349	46.073494	0.525421	3002
20	30	Other transport	Capital-intensive	-3.987821	14.377617	0.285292	724
1	11	Beverages	Labour-intensive	-3.659417	12.639491	0.439882	869
10	20	Chemicals	Labour-intensive	-1.821774	40.182327	0.578720	2941
13	23	Non-metallic minerals	Labour-intensive	-1.029061	22.295105	0.435097	4297
17	27	Electrical equipment	Capital-intensive	0.828607	27.808598	0.205108	2241
19	29	Motor vehicles	Labour-intensive	1.743288	37.477634	0.511520	2012
18	28	Machinery	Capital-intensive	2.737632	26.885536	0.238868	2672
14	24	Basic metals	Labour-intensive	3.176284	55.221661	0.681543	2616
16	26	Computer & electronics	Capital-intensive	4.185649	65.739418	0.143418	796
12	22	Rubber & plastics	Labour-intensive	4.500125	37.677013	0.453107	2634
6	16	Wood products	Capital-intensive	5.713552	34.592800	0.199390	1233
0	10	Food products	Capital-intensive	7.767037	15.075573	0.283512	6734
11	21	Pharmaceuticals	Labour-intensive	14.155688	20.947756	0.349006	1595
2	12	Tobacco	Capital-intensive	20.740908	-11.616011	0.249203	543

```
# ============================================================
# T-test:
# Did labour-intensive industries suffer larger declines?
# ============================================================

capital = df[df["group"] == "Capital-intensive"]["gva_drop_pct"]

labour = df[df["group"] == "Labour-intensive"]["gva_drop_pct"]

ttest = ttest_ind(
    capital,
    labour,
    equal_var=True
)

print(ttest)
```

```
TtestResult(statistic=np.float64(0.7175666761812546), pvalue=np.float64(0.48093118281118996), df=np.float64(21.0))
```

```
# ============================================================
# OLS regression
# ============================================================

df["labour_dummy"] = np.where(
    df["group"] == "Labour-intensive",
    1,
    0
)

X = sm.add_constant(df["labour_dummy"])

y = df["gva_drop_pct"]

model = sm.OLS(y, X).fit()

print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:           gva_drop_pct   R-squared:                       0.024
Model:                            OLS   Adj. R-squared:                 -0.023
Method:                 Least Squares   F-statistic:                    0.5149
Date:                Tue, 12 May 2026   Prob (F-statistic):              0.481
Time:                        09:14:05   Log-Likelihood:                -88.951
No. Observations:                  23   AIC:                             181.9
Df Residuals:                      21   BIC:                             184.2
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          -1.0872      3.496     -0.311      0.759      -8.357       6.182
labour_dummy   -3.6271      5.055     -0.718      0.481     -14.139       6.885
==============================================================================
Omnibus:                        3.566   Durbin-Watson:                   1.321
Prob(Omnibus):                  0.168   Jarque-Bera (JB):                1.930
Skew:                          -0.650   Prob(JB):                        0.381
Kurtosis:                       3.567   Cond. No.                         2.57
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

```
# ============================================================
# Alternative labour-intensity classification
# ============================================================

median_li = df["labour_intensity_baseline"].median()

df["alt_group"] = np.where(
    df["labour_intensity_baseline"] >= median_li,
    "Labour-intensive",
    "Capital-intensive"
)

robustness = df.groupby("alt_group")["gva_drop_pct"].mean()

robustness
```

alt_group	gva_drop_pct
Capital-intensive	-0.731295
Labour-intensive	-4.738376

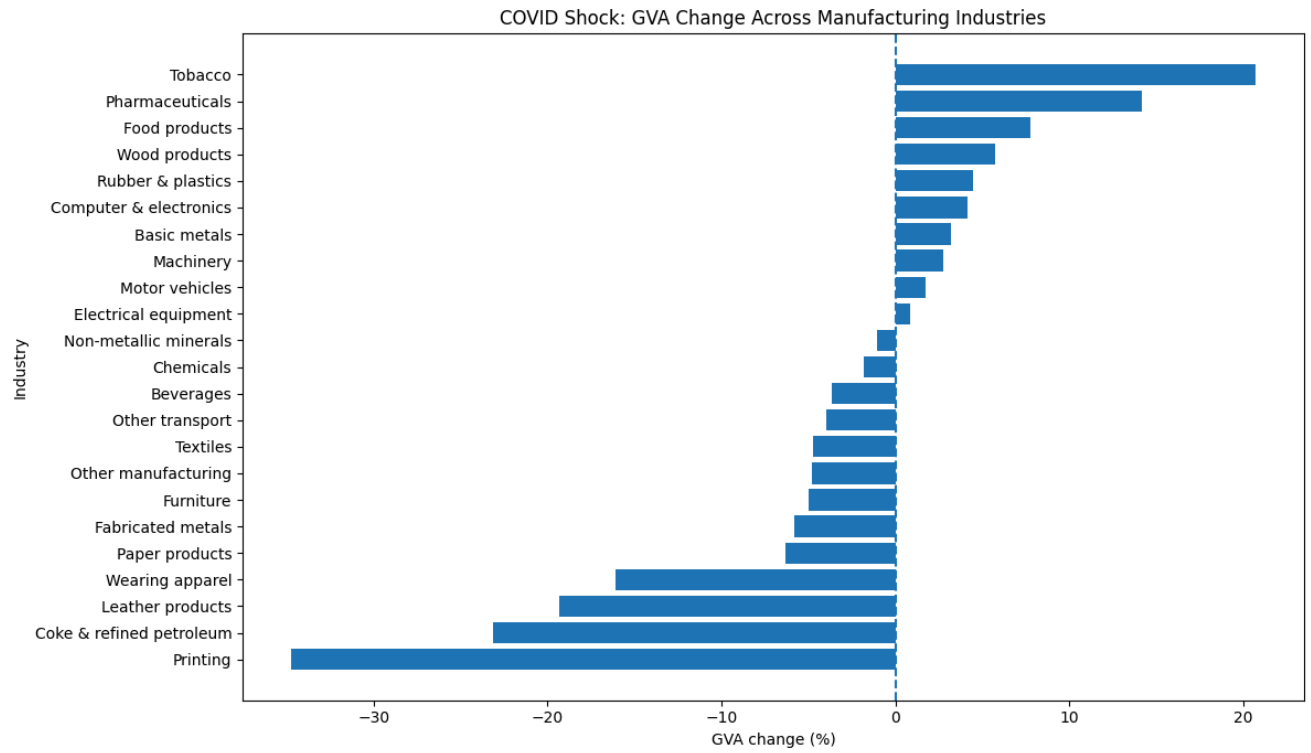
dtype: float64

```
# ============================================================
# Correlation matrix
# ============================================================

corr = df[[
    "gva_drop_pct",
    "gva_recovery_pct",
    "labour_intensity_baseline",
    "total_output20",
```

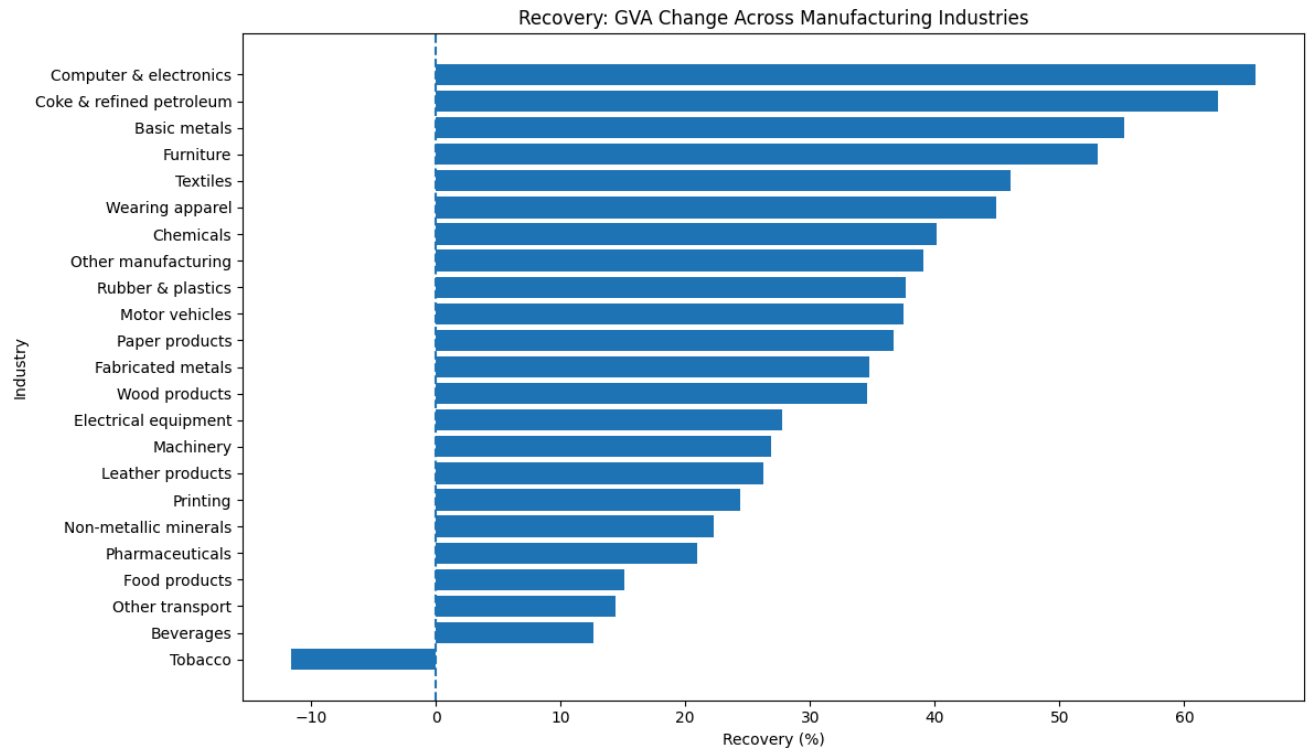
```
    "total_capital20"
]].corr()

corr.to_csv(
    "tables/correlation_matrix.csv"
)

corr
```

	gva_drop_pct	gva_recovery_pct	labour_intensity_baseline	total_output20	total_capital20
gva_drop_pct	1.000000	-0.326441	-0.297103	0.057906	0.287488
gva_recovery_pct	-0.326441	1.000000	0.381271	0.295053	0.156586
labour_intensity_baseline	-0.297103	0.381271	1.000000	0.670989	0.298115
total_output20	0.057906	0.295053	0.670989	1.000000	0.817580
total_capital20	0.287488	0.156586	0.298115	0.817580	1.000000

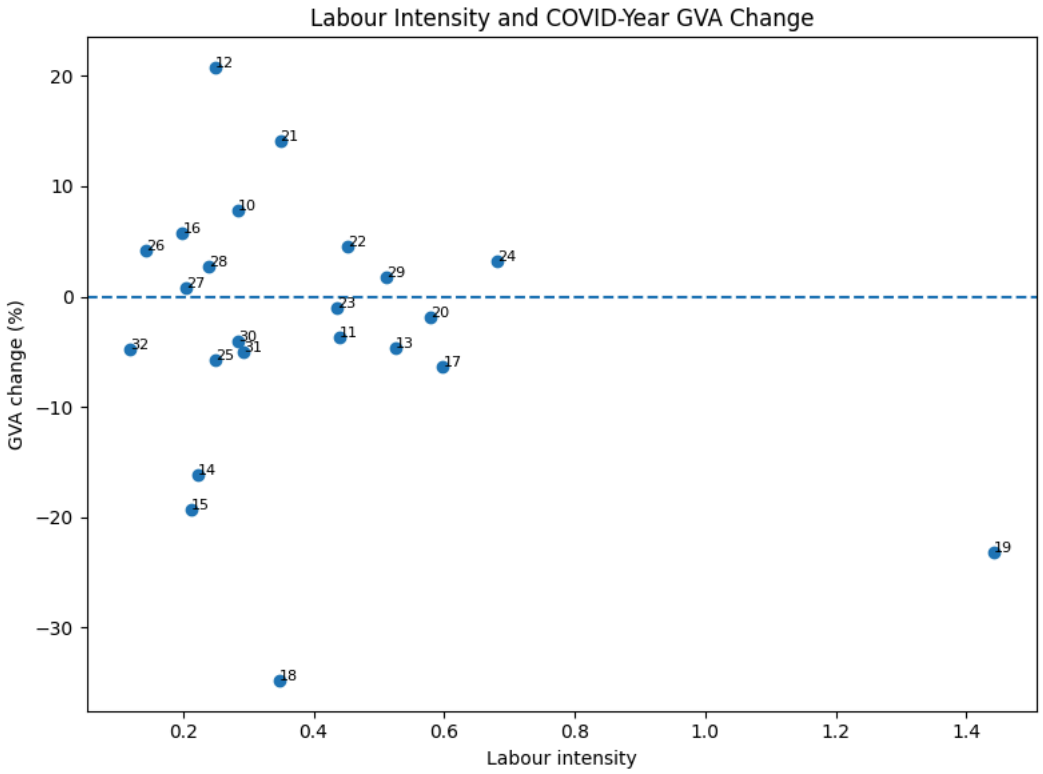
```
# ============================================================
# Industry GVA decline graph
# ============================================================

plot_df = ranking.copy()

plt.figure(figsize=(12, 7))

plt.barh(
    plot_df["industry_name"],
    plot_df["gva_drop_pct"]
)

plt.axvline(0, linestyle="--")

plt.xlabel("GVA change (%)")
plt.ylabel("Industry")

plt.title(
    "COVID Shock: GVA Change Across Manufacturing Industries"
)

plt.tight_layout()

plt.savefig(
    "figures/gva_drop_by_industry.png",
    dpi=300
)

plt.show()
```




```
recovery_df = df.sort_values("gva_recovery_pct")

plt.figure(figsize=(12, 7))

plt.barh(
    recovery_df["industry_name"],
    recovery_df["gva_recovery_pct"]
)

plt.axvline(0, linestyle="--")

plt.xlabel("Recovery (%)")
plt.ylabel("Industry")

plt.title(
    "Recovery: GVA Change Across Manufacturing Industries"
)

plt.tight_layout()

plt.savefig(
    "figures/gva_recovery_by_industry.png",
    dpi=300
)

plt.show()
```



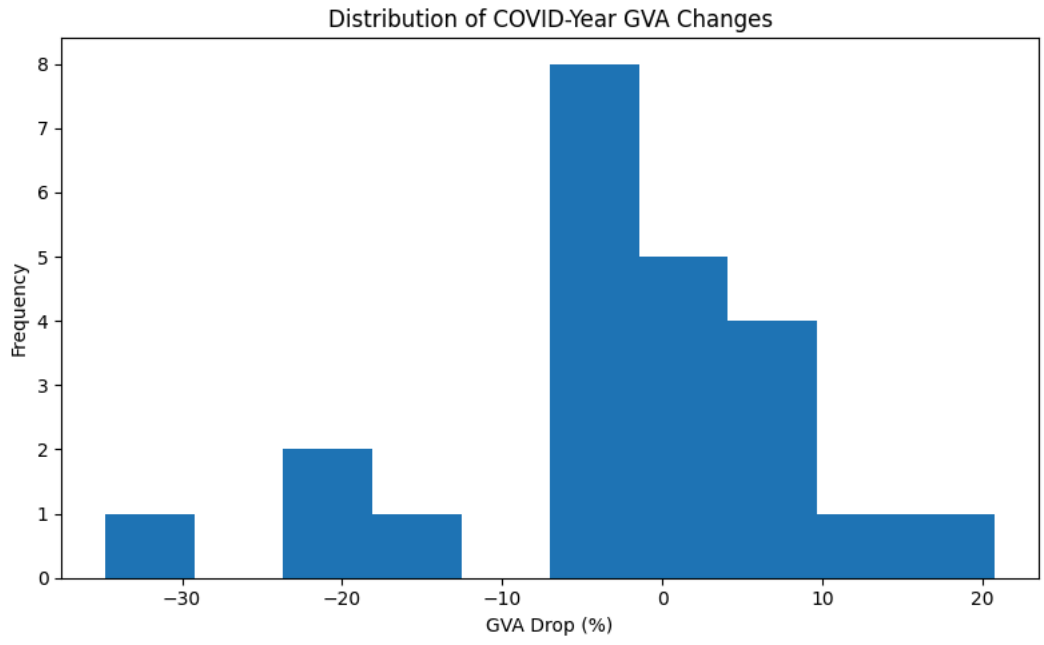

```
plt.figure(figsize=(8, 6))

plt.scatter(
    df["labour_intensity_baseline"],
    df["gva_drop_pct"]
)

for _, row in df.iterrows():
    plt.text(
        row["labour_intensity_baseline"],
        row["gva_drop_pct"],
        str(int(row["nic2"])),
        fontsize=8
    )

plt.axhline(0, linestyle="--")

plt.xlabel("Labour intensity")
plt.ylabel("GVA change (%)")

plt.title(
    "Labour Intensity and COVID-Year GVA Change"
)

plt.tight_layout()

plt.savefig(
    "figures/labour_intensity_vs_gva_drop.png",
    dpi=300
)

plt.show()
```




```
plt.figure(figsize=(7, 5))

df.boxplot(
    column="gva_drop_pct",
    by="group"
)

plt.title("GVA Drop by Industry Type")
plt.suptitle("")

plt.ylabel("GVA Drop (%)")

plt.tight_layout()

plt.savefig(
    "figures/gva_drop_boxplot.png",
    dpi=300
)

plt.show()
```

```
<Figure size 700x500 with 0 Axes>
```


```
plt.figure(figsize=(8, 5))

plt.hist(
    df["gva_drop_pct"],
    bins=10
)

plt.xlabel("GVA Drop (%)")
plt.ylabel("Frequency")

plt.title(
    "Distribution of COVID-Year GVA Changes"
)

plt.tight_layout()

plt.savefig(
    "figures/gva_drop_distribution.png",
    dpi=300
)

plt.show()
```



```
# ============================================================
# Random Forest:
# exploratory pattern discovery
# ============================================================

features = [
    "labour_intensity_baseline",
    "total_gva20",
    "total_output20",
    "total_labour_cost20",
    "total_capital20",
    "factory_count20",
    "gva_per_labour_cost20",
    "capital_gva_ratio20"
]

model_df = df.dropna(
    subset=features + ["gva_drop_pct"]
).copy()

X = model_df[features]

y = model_df["gva_drop_pct"]

rf = RandomForestRegressor(
    n_estimators=500,
    random_state=42,
    min_samples_leaf=2
)

rf.fit(X, y)

pred = rf.predict(X)

r2 = r2_score(y, pred)

mae = mean_absolute_error(y, pred)

print("R2:", r2)
print("MAE:", mae)
```

```
R2: 0.6445916445787929
MAE: 4.976615697882584
```

```
importances = pd.DataFrame({
    "feature": features,
    "importance": rf.feature_importances_
}).sort_values(
    "importance",
    ascending=False
)

importances.to_csv(
    "tables/random_forest_feature_importance.csv",
    index=False
)

importances
```

	feature	importance
5	factory_count20	0.232564
6	gva_per_labour_cost20	0.200025
7	capital_gva_ratio20	0.147641
2	total_output20	0.113808
4	total_capital20	0.111004
1	total_gva20	0.076027
0	labour_intensity_baseline	0.070579
3	total_labour_cost20	0.048353

```
plt.figure(figsize=(9, 5))

plt.barh(
    importances["feature"],
    importances["importance"]
)
```